



> INITIATING SEQUENCE...

CLOUD-NATIVE DECEPTION ARCHITECTURE

TARGET: FINANCIAL SECURITY & THREAT INTEL

OPERATOR

Mojiz Zaidi
(2311051806)

SUPERVISOR

Prof. Ugur Ilker Atamca

DATE

JAN 04 2026

THE PERIMETER PARADOX

LEGACY SYSTEMS

- > Defined Perimeter
- > Physical Firewalls ☐
- > Static IP Ranges

STATUS: OBSOLETE

CLOUD REALITY

- > Identity is the new perimeter
- > Walls are lines of code (IaC)
- > Firewalls cannot stop stolen credentials

RISK: HIGH

> ALERT: Traditional defenses fail against API abuse and Identity Theft.

FLIPPING THE **ASYMMETRY**

PASSIVE DEFENSE

Defender success required: 100%

Attacker success required: 1%



ACTIVE DECEPTION

Attacker success required: 100%

One mistake (touching a decoy) = GAME OVER



CONCEPT: "LEGITIMACY IS KNOWLEDGE"

"How do we distinguish a user from a hacker without AI?"



The Employee

Has Bookmarks. Knows the DNS. Goes straight to target.



The Attacker

Scans IP ranges. Guesses Subdomains. Uses leaked keys.

> CONCLUSION: Any interaction with a hidden decoy is PROOF OF MALICE.

OUT-OF-BAND ARCHITECTURE

1. MANAGEMENT PLANE

Restricted Jump Box. SSH access only. Isolated.

2. DECEPTION PLANE (THE TRAP)

Contains T-Pot Sensors & Fake Financial Web App. NO ROUTE TO PRODUCTION.

3. OBSERVABILITY PLANE (THE EYES)

WORM Storage (Write Once, Read Many). Logs cannot be deleted by attackers.



AZURE IMPLEMENTATION

```
root@azure-vm:~# ./deploy_decoy.sh

> Initializing Environment...

> Platform:

> Loading Network Security Groups...

> Starting T-Pot Sensors (SSH, Telnet)...

> Deploying Financial Flask App...

> Hiding DNS Records...

> System Ready. Trap is live.
```



THE STACK

- > **Platform:** Azure (Cost/Scale)
- > **T-Pot:** Catches bots & scripts.
- > **Custom App:** Python/Flask simulating a Bank Login & Transfer API.



INTERACTIVE SCENARIO

You are the attacker. You find a leaked AWS Key on
GitHub.

```
DB.AWS_ACCESS_KEY_ID = "AKIA..."
```

> DECISION REQUIRED:

- A) Connection Refused.
- B) Login to Production
- C) Silently Routed to Decoy

UNISEX CONSISTENCY MODEL

THE "SPLIT-BRAIN" PROBLEM

Smart hackers check for consistency. If they find a password in a file, but it doesn't work on the DB, they know it's a trap.

THE SOLUTION

Narrative Integrity: Information found in Layer A *must* work in Layer B.



Result: Attacker stays engaged.

ENGAGEMENT METRICS

DWELL TIME (TIME IN SYSTEM)



0%

False Positives

7X

Longer Engagement

SYSTEM CONCLUSION

- > **Perimeter is Dead:** Switch from blocking to strategic engagement.
- > **Zero False Positives:** "Legitimacy is Knowledge".
- > **Asset Transformation:** Attackers become intelligence sources.

FUTURE WORK:

Integration with ML for dynamic lure generation.

